Predicting Errors in MLB Umpire Calls using Bayesian Models'

Preston K, Fletcher N, Miguel C, Eric T.

Department of Industrial Systems Engineering, Texas A&M University

ISEN 427: Decision and Risk Analysis

Dr. Alfredo Garcia

May 06, 2025

Predicting Errors in Umpire Calls using Bayesian Models'

Introduction:

Baseball is a popular sport around the world, and in the United States, it is governed by Major League Baseball (MLB). One of the most important roles in the game is played by the home plate umpire, who decides if each pitch is a ball or a strike. These decisions happen quickly and can significantly impact the game. In this project, we study the accuracy of these calls by using data from the PITCHf/x tracking system, which records the exact location of each pitch. We will focus on three umpires and examine how often their calls align with the actual strike zone. We built several models using different combinations of features like pitch location, zone, pitch type, batter stance, pitcher handedness, and game situation to determine which factors most affect an umpire's accuracy. By analyzing model results and predictions, we compare their performance and understand each variable's impact. This helps us appreciate how Bayesian methods can model human judgment in sports.

Preprocessing the data:

A lot of the data in this dataset is stored as strings or objects, and before they can be used for training our PyMC models, these features must be properly cleaned. Features that are stored as strings or objects and which we wish to analyze need to be specifically processed into numerical features before we can effectively build the models. Throughout the preprocessing phase, all encoding methods and preprocessing techniques were applied to all three datasets. This ensures that any noticeable deviation in results among the three different umpires is due to the features themselves, rather than differences in our training data 's preprocessing.

The most prominent feature we needed to clean was the type of pitch thrown. We chose to encode pitch_type by creating binary indicators for which category the pitch falls under, first, we grouped the various pitch types into 4 distinct categories. 'FF', 'FA', 'FT', 'SI', and 'FC' (4 seam, 4 seam again, two seam, and cutter) were grouped into "fastballs". 'CH', 'EP', 'FO', and 'FS' (Change, ephus, fork, and split) were grouped into "changeups". 'SL', 'CU', 'ST', 'SV', 'CS' (Slider, curve, sweeper, slurve, and slow curve) were grouped into "off_speed". Finally, 'KC', 'KN', 'SC' (Knuckle curve, knuckle ball, and screwball were grouped into 'rare'. 'AB' and 'AS' (automatic ball and strike) are not included in the specialization, as these are circumstantial and aren't determined specifically by the pitcher (for example, a pitch clock violation occurs against the pitcher or the hitter, and an automatic strike is called). We then applied one-hot encoding on

the remaining groups to get indicators as to what the pitch type was. Similarly, we encoded a new feature full count as a binary indicator whether there are 3 balls and 2 strikes at the pitch.

In this project, the variable that we are trying to predict is the "error_in_decision". In our dataset, this feature is listed as a categorical variable with 2 different entries, "correct" and "incorrect". For simplicity, we chose to apply a binary encoder to the feature, encoding 0 to entries where the call is correct, and 1 if an error was made in the decision.

```
def clean_df(orig_df):
    """
    Pitch types:
        ['SI' 'AB' 'SL' 'FC' 'CH' 'CU' 'FF' 'FS' 'EP' 'KC' 'FO' 'KN' 'SC' 'ST'
         'SV' 'CS' 'FA']
    Handedness: R=1, L=0
    Description: 'ball': 0, 'called_strike': 1
    Error in decision: 'correct': 0, 'incorrect': 1
    """
    df = orig_df.copy()
    # Drop nulls
    df = df.dropna()

    # Map pitch types to binary columns
    unique_pt = df['pitch_type'].unique()
    fastballs = ['FF', 'FA', 'FT', 'SI', 'FC'] # 4 seam, 4 seam again, two seam, sinker, cutter
    changeups = ['CH', 'EP', 'FO', 'FS'] # Change, ephus, fork, split
    off_speed = ['SL', 'CU', 'ST', 'SV', 'CS'] # Slider, curve, sweeper, slurve, slow curve
    rare = ['KC', 'KN', 'SC'] # Knucle curve, knuckle ball, screwball
    # AB and AS are 'automatic ball/strike' so they are not included. This happens when something like a pitch clock violation occurs
    # Against the pitcher or the hitter

    # Create new binary columns
    df['fastball'] = df['pitch_type'].isin(fastballs).astype(int)
    df['changeup'] = df['pitch_type'].isin(changeups).astype(int)
    df['off_speed'] = df['pitch_type'].isin(off_speed).astype(int)
    df['rare'] = df['pitch_type'].isin(rare).astype(int)

    # Drop ptich_type column since it is not categorical
    df = df.drop(columns='pitch_type')

    # Same with error
    df['error_in_decision'] = df['error_in_decision'].map({'correct': 0, 'incorrect': 1})

    # Fix handedness
    df['stand'] = df['stand'].map({'R': 1, 'L': 0})
    df['p_throws'] = df['p_throws'].map({'R': 1, 'L': 0})

    df['description'] = df['description'].map({'ball': 0, 'called_strike': 1})

    df['full_count'] = ((df['balls'] == 3) & (df['strikes'] == 2)).astype(int)


    # Max min scale the data to make coefficienst more interpretable
    columns_to_scale = df.drop(columns=['error_in_decision', 'fastball', 'changeup', 'off_speed', 'rare']).columns
    scaler = StandardScaler()
    df[columns_to_scale] = scaler.fit_transform(df[columns_to_scale])

    return df
```

Figure 1: Function used to apply the pre-processing techniques described above.

Choosing our Features:

The data that was collected was based on pitches made at the MLB professional level. The umpires who made these calls are very well trained and are not prone to making simple errors like those at the amateur level. Based on our prior domain knowledge, we figure that the factors that give the highest chance for the umpire to make an error in the call are the pitch location, whether there is a full count (2 strikes and 3 balls), the change in run expectancy, whether the pitch was a fast ball, or the total number of plate appearances.

Pitch location tells us the Euclidean distance from the ball to the strike zone. We hypothesize that the smaller the distance is to the strike zone the chance that the umpire will make an error in the call will increase. Since the strike zone is not explicitly laid out to the umpire during the game. If the distance to the strike zone is not very large, the umpire might accidentally be swayed one way or the other. In a particular "at bat", if there is a full count, the umpire may be more fatigued from being stuck in the pitch zone.

We included the change in run expectancy because it reflects how important a pitch is in the context of the game. When a lot is at stake, such as when runners are on base or the inning is close, the pressure may subtly influence the umpire's decision. This feature could help us understand how game situations might affect their calls.

If the pitch type was a fastball, especially at high speeds, the ball might be traveling so fast that the chance of an umpire making a bad call increases. Finally, the pitch number indicates the total number of pitches made in a particular game. The higher the number is, we hypothesize that as the pitch number increases, the likelihood of the umpire making a bad call will also increase.

Choosing our Prior Distributions:

Prior distributions were chosen based on domain knowledge of the game of baseball. For pitch location, smaller values indicate pitches that were closer to the edge of the strike zone. Basic intuition tells us that it is much more likely for umpires to misidentify borderline pitches, implying a negative relationship between pitch location and the error in the decision binary variable in our pitch decision model. Because of this, a negative half-normal prior distribution was chosen for the pitch location parameter. The check on this prior using data from umpire 1 can be seen below in **Figure 2**. This same intuition was applied to the full count and delta run expectancy variables, using the same negative half-normal distribution for both. An umpire will become more focused in a full count scenario because of the importance of the situation. The same can be said when the call makes a big difference in a game and causes a large change in run expectancy.

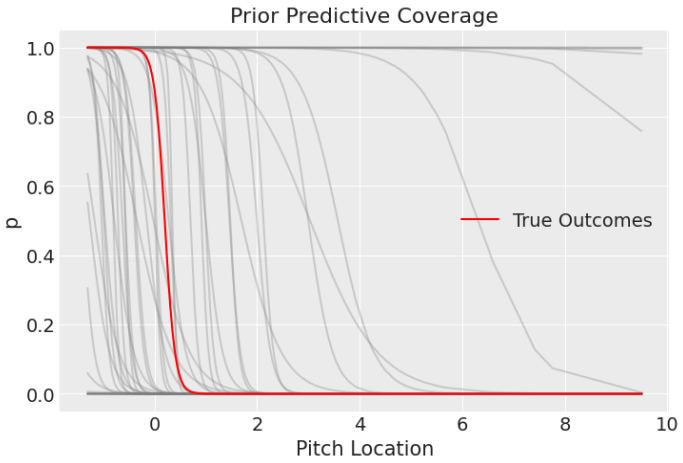

Figure 2. Example of a prior check on the pitch location variable in the umpire 1 logistic model. The prior distribution being used is a negative half-normal.

For the binary fastball variable and pitch count variable, a positive half-normal distribution was originally applied. When applied with the model, the positive half-normal fastball distribution was found to fit very well, which aligns with the expectation that balls thrown with greater velocity are harder to see and identify correctly. However, when applying this distribution to the pitch count model, it was found that the distribution did not fit the data very well. The parameter appeared to be pushing to the negative side. The original thought was that an umpire would become tired throughout a long at-bat, but this was proved to be the opposite of the truth. Through this prior check, we discovered that the umpires become more accurate as an at-bat goes on. The pitch count prior was updated to a negative half-normal, which gave much better results.

Results

Umpire 1

Using the features and the priors described in the previous sections, we first trained 5 PyMC models (each model containing only 1 feature) using the Umpire 1 dataset. Following the training, we generated the posterior distribution for our values. Each model was then evaluated using its F1 score, which we stored along with its corresponding feature. By plotting the F1 scores in a bar chart with each feature's model, we obtain:

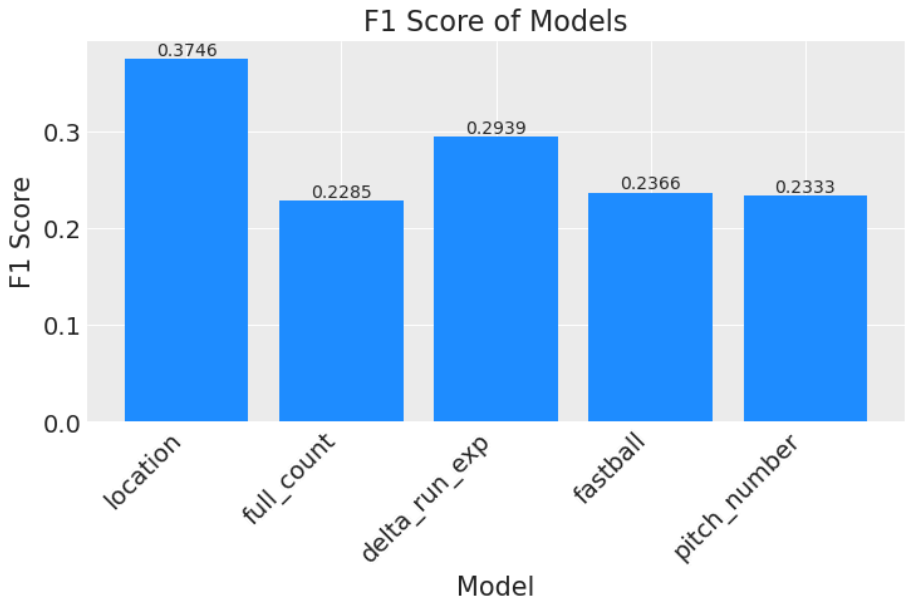

Figure 3. F1-scores with 1 feature model

Observing the output plot of f1-scores, we can see that pitch location yields the highest f1-score of 0.3746. This implies that the single model with only pitch location can predict whether umpire 1 will make a bad call more than the other models. Notice that full_count has the lowest F1-score out of the 5 (with an F1-score of only 0.2285). This means that this model has the lowest predictive power, and we will omit it for the rest of this analysis. This leaves us with pitch location, delta_run_exp, fastball, and pitch number.

Now we will train models with different combinations of the remaining features. Some combinations we made were pitch_loc and delta_exp, delta_exp and fastball, pitch_loc and pitch_number, pitch_loc and full_count, 3 features (pitch_loc, fastball, and deltarun_exp), and all 4 features (pitch_loc, delta_run_exp, fastball, and pitchnum). After building these models and generating their respective posteriors. We once again evaluate each model's F1-score and generate a plot with all of them.

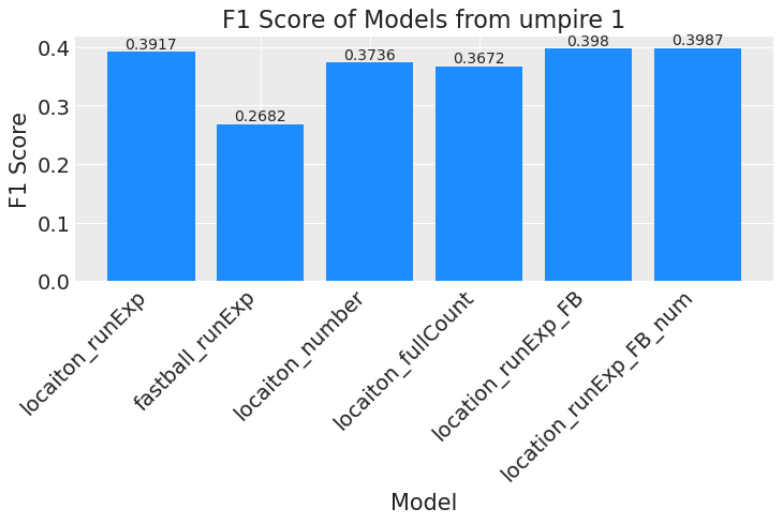

Figure 4. F1-Scores from a combination of features

Observing the above plot, the model with all 4 features yielded the highest f1-score of 0.3987, with the model only containing pitch location, delta run exp, and fastball had an almost identical f1-score of 0.398 with the model containing pitch location and delta run exp having a slightly lower f1-score of 0.3917. This means that either of these models yields a similar probability of umpire 1 making a bad call, but the full model is the best.

Umpire 2

The first order of process with the second umpire was also to plot the posteriors for each parameter that were chosen: location, full count, change in run expectancy, fastball, and pitch number. After gathering that information, we were able to get the accuracy of our posteriors by making predictions and then calculating the accuracy with the actual data that was provided. In the end, the individual F1 scores were plotted in Figure 5.

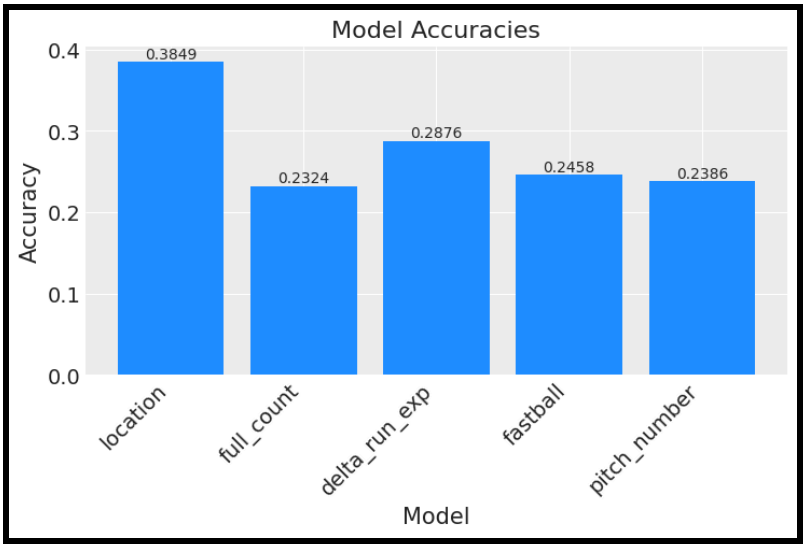

Figure 5. Model Accuracy for parameters for Umpire 2.

Figure 5 shows the F1 scores for each parameter. Among them, location stood out with the highest F1 score of 0.3849, followed by change in run expectancy with a score of 0.2876. The remaining parameters, full count, fastball, and pitch number, produced relatively similar and lower F1 scores in the low-to-mid 0.20s range.

After starting with the results with the F1 scores of each parameter, we then started combining different parameters and seeing the F1 score of different combinations as we saw fit. The first combination we did was location and change in run expectancy, as they yielded the highest accuracy individually. This yielded a F1 score of 0.4036 as seen in Figure 6. So next in line, we combined fastball and run expectancy as our second and third highest individual F1 scorers. However, as seen again in Figure 5, this yielded a lower score of 0.2628. So, from that accuracy check, we decided to use pitch location in all of our remaining combinations.

We continued combining location with other features: first with pitch number, then with full count, and finally with fastball. Among all combinations, the best performing was location, run expectancy, fastball, and full count, which reached the highest F1 score of 0.4081 (see Figure 6). Interestingly, the improvement over using just location and run expectancy was marginal, suggesting that these two features are the primary drivers of accuracy, while others offer diminishing returns.

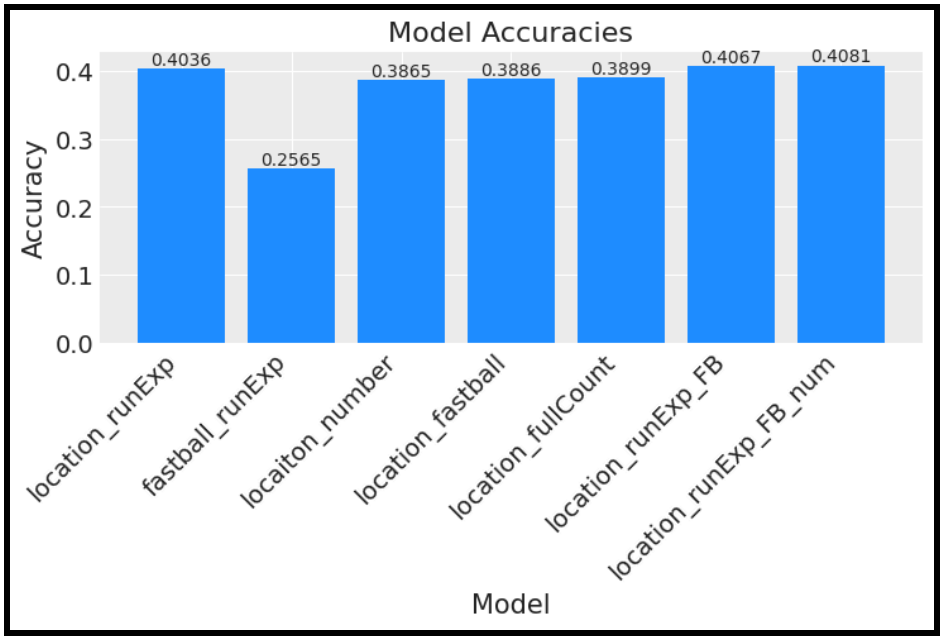

Figure 6. Model accuracies of different parameter combinations for umpire 2.

This analysis implies that while combining more parameters can offer small gains, location and change in run expectancy alone provide strong predictive performance, making them valuable inputs for modeling umpire decision-making.

Umpire 3

Similar to the previous 2 umpires, we generated base models based on the chosen parameters above and evaluated the model performance based on the F1-score. The plot of the individual F1-scores is.

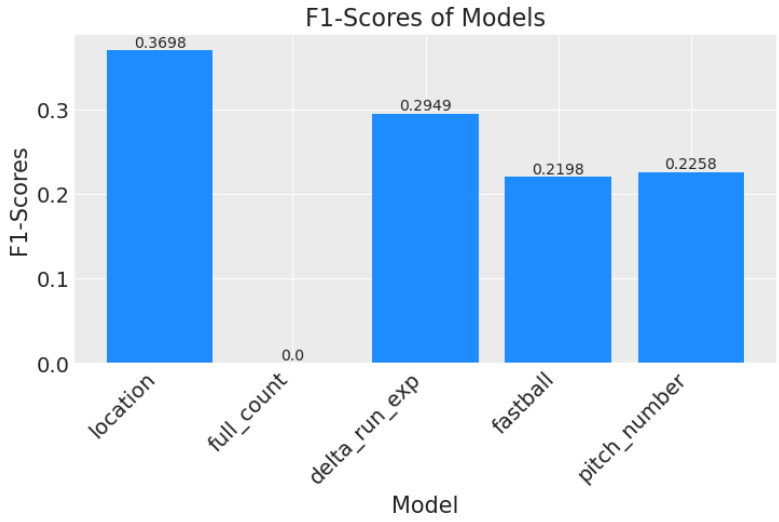

Figure 6: Plot of 1-Feature model F1-Scores

From this plot, we can see that the model built with the pitch_location feature yielded the highest F1-score (0.3690) amongst the 5 chosen plots by a significant margin (the second-best model yielded an F1-score of 0.2949). This indicates that, as of right now, it is the best model to use when trying to predict whether umpire 3 will make a bad call. Notice that full_count yields an F1-score of 0, so for future analysis, we will omit this feature as it has no predictive power in this situation.

Following the building of the single-feature models, we started testing models with a combination of the remaining 4 chosen features. Some combinations we made were pitch_loc and delta_exp, delta_exp and fastball, pitch_loc and pitch_number, pitch_loc and full_count, 3 features (pitch_loc, fastball, and deltarun_exp), and all 4 features (pitch_loc, delta_run_exp, fastball, and pitch num). After evaluating the posteriors and similarly finding the F1-scores as the 1-feature models, plotting the F1-scores yields.

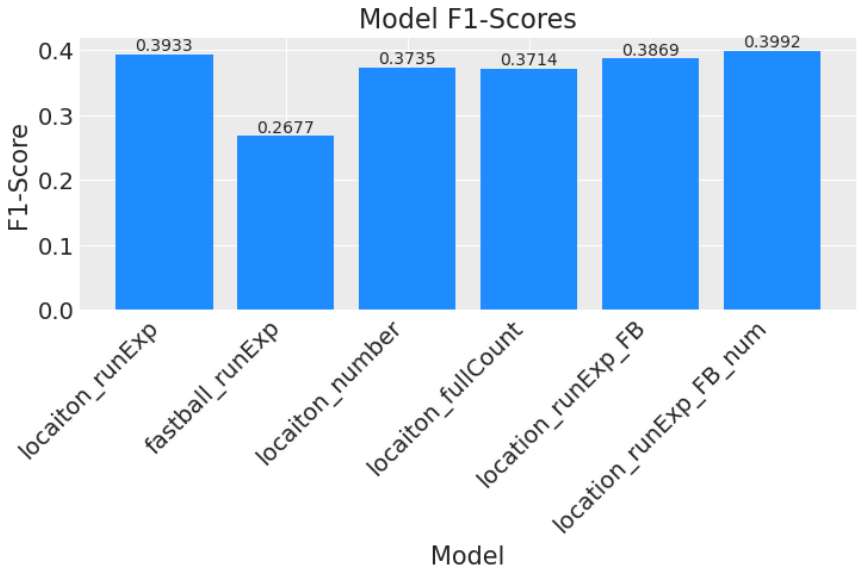

Figure 7. Plot of combined models' F1-scores for umpire 3

From the above figure, we can see that the models with all 4 features and the model with only pitch location and delta run expected yielded very similar results in terms of f1-score (approx 0.3992), with the other models yielding similar, but not as good, f1-scores. This indicates that we can either choose the full stack model or the simplified model with only those 2 features, and it would yield very similar predictions in umpire 3's probability of a bad call.

Conclusions

Through our modeling and analysis of the umpires' decision-making, we found that pitch location is the strongest single predictor of whether a pitch is called a strike or a ball. However, by combining location with additional context-based features, particularly change in run expectancy, we achieved improved predictive performance. Our best-performing model in all three umpires used a combination of location, run expectancy, fastball, and full count. While this improvement was modest, it suggests that game context does influence umpire behavior, though not as strongly as pitch location. Overall, our results highlight the complexity of human decision-making in sports and demonstrate that a combination of physical and situational factors offers the best insight into umpire performance.

Resources:

GitHub Repository: https://github.com/PRESTONKOU12/ISEN427-Baseball-Project